

Spring Boot

Comprehensive Training Notes

From Fundamentals to Production-Ready Applications

Prepared for Professional Training

Includes Interview Preparation Questions & Real-World Examples

Table of Contents

1. Introduction to Spring Boot

1.1 What is Spring Boot?

Spring Boot is an open-source, Java-based framework built on top of the Spring Framework that dramatically simplifies the process of creating production-ready, stand-alone Spring applications. Developed and maintained by Pivotal (now VMware Tanzu), Spring Boot was first released in April 2014 with the explicit goal of eliminating the boilerplate configuration that had long been a pain point for Spring developers.

Before Spring Boot, developers had to write extensive XML configuration files, manage complex dependency hierarchies, and spend significant time setting up the infrastructure before writing a single line of business logic. Spring Boot changed this paradigm entirely by introducing the concept of "convention over configuration" — sensible defaults that work out of the box while still allowing full customization when needed.

Real-World Analogy: Think of Spring Boot like buying a pre-configured laptop versus building a PC from scratch. With a traditional Spring application, you are choosing every component (motherboard, RAM, GPU, power supply) and assembling them yourself. With Spring Boot, you get a fully assembled, ready-to-use machine. You can still upgrade individual components later, but you can start working immediately.

Interview Q: What is Spring Boot and why was it created?

A: Spring Boot is an opinionated framework built on top of the Spring Framework that simplifies the development of production-grade Spring applications. It was created to address the complexity of traditional Spring configuration by providing auto-configuration, embedded servers, starter dependencies, and production-ready features out of the box. It follows the principle of convention over configuration.

1.2 Spring vs Spring Boot — Key Differences

Understanding the distinction between the Spring Framework and Spring Boot is fundamental. The Spring Framework is a comprehensive programming and configuration model for enterprise Java applications. Spring Boot, on the other hand, is a layer on top of Spring that streamlines setup and deployment.

Aspect	Spring Framework	Spring Boot
Configuration	Manual XML or Java-based config	Auto-configuration with sensible defaults
Server Setup	Requires external server (Tomcat, JBoss)	Embedded server (Tomcat, Jetty, Undertow)

Aspect	Spring Framework	Spring Boot
Dependency Management	Manual dependency version management	Starter POMs with curated dependencies
Deployment	WAR file to external server	Executable JAR with embedded server
Boilerplate Code	Significant setup required	Minimal — ready to run in minutes
Production Features	Manual setup for metrics, health	Built-in via Spring Boot Actuator
Learning Curve	Steep for beginners	Gentle — get started quickly
Opinionated?	No — full flexibility	Yes — defaults provided, overridable

Interview Q: What is the difference between Spring and Spring Boot?

A: Spring is the core framework providing IoC, DI, AOP, and a programming model for Java enterprise applications. Spring Boot is built on top of Spring and provides auto-configuration, embedded servers, starter dependencies, and production-ready features. Spring requires extensive manual configuration; Spring Boot offers opinionated defaults to get applications running quickly with minimal setup. Spring Boot does NOT replace Spring — it enhances it.

1.3 Key Pillars of Spring Boot

Spring Boot is built around four foundational pillars that together eliminate the traditional pain points of Spring application development:

- 1. Auto-Configuration:** Spring Boot automatically configures your application based on the JAR dependencies present in the classpath. If you have `spring-boot-starter-web` on the classpath, Spring Boot automatically configures an embedded Tomcat server, a `DispatcherServlet`, default error pages, and sensible Jackson message converters for JSON. You can override any auto-configuration by providing your own bean definitions.
- 2. Starter Dependencies:** Starters are curated sets of dependencies grouped by purpose. Instead of manually adding 10–15 individual dependencies for a web application, you add a single `spring-boot-starter-web`. This eliminates version conflicts and ensures compatibility. Think of starters as pre-built ingredient kits for specific recipes.
- 3. Embedded Server:** Spring Boot embeds a servlet container (Tomcat by default) directly into your application JAR. This means your application is completely self-contained — no need to install and configure an external application server. You simply run `java -jar myapp.jar` and the server starts automatically.
- 4. Production-Ready Features:** Through Spring Boot Actuator, your application automatically gets health checks, metrics, environment information, HTTP tracing, thread dumps, and more. These endpoints are

essential for monitoring applications in production and integrating with tools like Prometheus, Grafana, and ELK Stack.

Interview Q: What are the main features/pillars of Spring Boot?

A: The four main pillars are: (1) Auto-Configuration — automatically configures beans based on classpath dependencies, (2) Starter Dependencies — curated, pre-tested dependency sets that simplify Maven/Gradle configuration, (3) Embedded Server — includes Tomcat/Jetty/Undertow inside the JAR for self-contained deployment, and (4) Production-Ready Features — Actuator provides health checks, metrics, auditing, and monitoring endpoints out of the box.

1.4 Spring Boot Architecture Overview

Spring Boot follows a layered architecture pattern that promotes separation of concerns. Understanding this architecture is critical both for building well-structured applications and for interview discussions.

Presentation Layer (Controller): Handles HTTP requests and responses. Controllers are annotated with `@RestController` or `@Controller` and define API endpoints using `@RequestMapping`, `@GetMapping`, `@PostMapping`, etc. This layer should contain no business logic — it simply delegates to the service layer.

Business Layer (Service): Contains the core business logic of the application. Service classes are annotated with `@Service` and are where you implement calculations, validations, transformations, and orchestration of data. Services are injected into controllers via dependency injection.

Persistence Layer (Repository): Manages database operations. Repository interfaces extend `JpaRepository` or `CrudRepository` and provide CRUD operations. Spring Data JPA auto-implements these interfaces at runtime. Annotated with `@Repository`.

Database Layer: The actual database (MySQL, PostgreSQL, H2, MongoDB, etc.). Spring Boot auto-configures the `DataSource` based on your `application.properties` and the database driver on the classpath.

Best Practice



Always maintain strict layer separation. Controllers should never directly access repositories. This makes your code testable, maintainable, and follows the Single Responsibility Principle.

2. Setting Up a Spring Boot Project

2.1 Spring Initializr

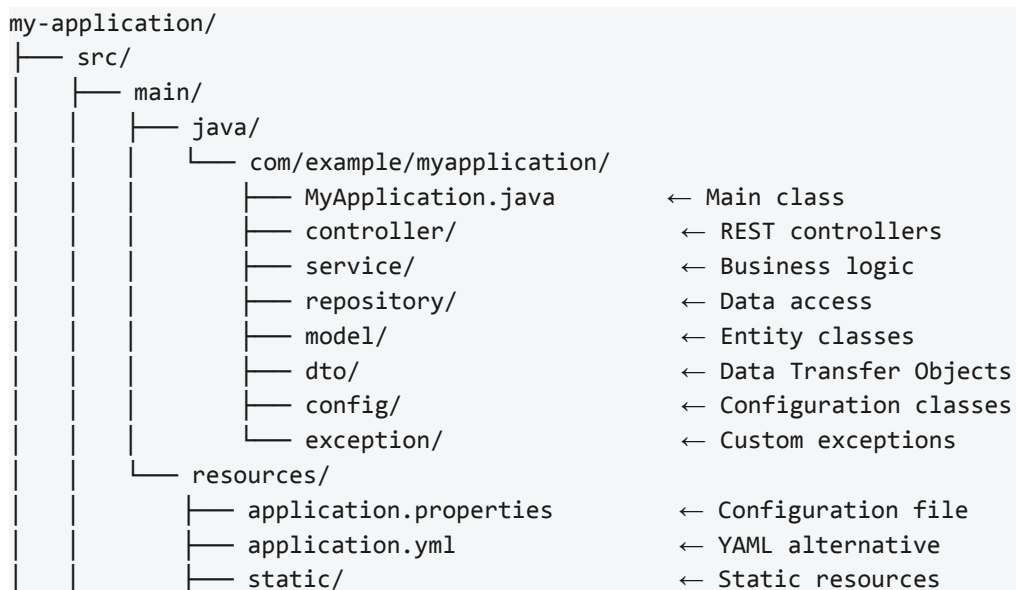
Spring Initializr (<https://start.spring.io>) is the official project scaffolding tool for Spring Boot applications. It generates a complete project structure with the selected build tool, language, Spring Boot version, and dependencies. Most IDEs (IntelliJ IDEA, Eclipse with STS, VS Code) have built-in integration with Spring Initializr.

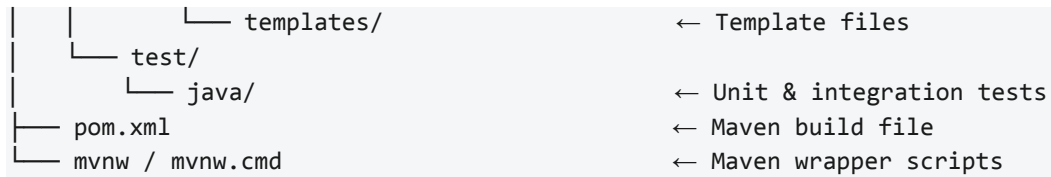
When creating a project, you configure the following:

Field	Description	Common Choices
Project Type	Build tool to use	Maven Project or Gradle Project
Language	Programming language	Java, Kotlin, Groovy
Spring Boot Version	Framework version	Latest stable (e.g., 3.x.x)
Group	Maven group ID (reverse domain)	com.example, com.mycompany
Artifact	Project name / artifact ID	my-application
Packaging	Application packaging type	Jar (recommended) or War
Java Version	JDK version	17, 21 (LTS versions recommended)

2.2 Project Structure

A Spring Boot project generated by Spring Initializr follows a standard Maven/Gradle directory layout. Understanding this structure is essential for organizing your code correctly.



**Important Note**

The main application class (annotated with `@SpringBootApplication`) MUST be placed in the root package (e.g., `com.example.myapplication`). All other packages must be sub-packages of this root. Spring Boot uses component scanning starting from the main class package, so if your main class is in `com.example.myapplication`, it will scan `com.example.myapplication.controller`, `com.example.myapplication.service`, etc. Placing a class in `com.example.other` will NOT be scanned.

2.3 The Main Application Class

Every Spring Boot application has a single entry point — the main application class. This class is annotated with `@SpringBootApplication` and contains the standard Java `main()` method.

```

package com.example.myapplication;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class MyApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}

```

`@SpringBootApplication` is a convenience annotation that combines three essential annotations:

Annotation	Purpose
<code>@SpringBootConfiguration</code>	Marks this class as a configuration class (specialized <code>@Configuration</code>). Allows the class to register beans via <code>@Bean</code> methods.
<code>@EnableAutoConfiguration</code>	Enables Spring Boot's auto-configuration mechanism. It tells Spring Boot to automatically configure beans based on classpath dependencies, property settings, and other factors.
<code>@ComponentScan</code>	Enables component scanning in the current package and all sub-packages. This discovers classes annotated with <code>@Component</code> , <code>@Service</code> , <code>@Repository</code> , <code>@Controller</code> , etc.

Interview Q: What does @SpringBootApplication do internally?

A: @SpringBootApplication is a meta-annotation that combines @SpringBootConfiguration (marks the class as a configuration source), @EnableAutoConfiguration (triggers Spring Boot's auto-configuration based on classpath and properties), and @ComponentScan (scans the current package and sub-packages for Spring components). It is the starting point of every Spring Boot application.

2.4 The pom.xml Explained

The pom.xml is the heart of a Maven-based Spring Boot project. It defines the project metadata, parent configuration, dependencies, and build plugins.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>

  <!-- Spring Boot Parent POM: manages dependency versions -->
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.0</version>
  </parent>

  <groupId>com.example</groupId>
  <artifactId>my-application</artifactId>
  <version>1.0.0</version>

  <properties>
    <java.version>17</java.version>
  </properties>

  <dependencies>
    <!-- Web Starter: Tomcat + Spring MVC + Jackson -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <!-- Test Starter: JUnit 5 + Mockito + Spring Test -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
```

```
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

Why No Version Numbers on Dependencies?



When you use spring-boot-starter-parent as the parent POM, it provides dependency management that pre-defines compatible versions for hundreds of libraries. That is why you do not need to specify <version> for Spring Boot starters and many common libraries. This is one of Spring Boot's greatest strengths — it eliminates version conflicts.

3. Auto-Configuration Deep Dive

3.1 How Auto-Configuration Works

Auto-configuration is the magic behind Spring Boot's simplicity. When your application starts, Spring Boot examines the classpath, existing bean definitions, and property settings to automatically configure the application context. This process follows a well-defined mechanism.

Step 1 — Classpath Scanning: Spring Boot checks which JAR files are on the classpath. For example, if `spring-webmvc.jar` is present, it knows you want a web application.

Step 2 — Loading Auto-Configuration Classes: Spring Boot reads the `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` file (in Spring Boot 3.x) or `META-INF/spring.factories` (in 2.x) to find all auto-configuration classes.

Step 3 — Conditional Evaluation: Each auto-configuration class uses `@Conditional` annotations to determine if it should be applied. If the conditions are met, the beans are created; otherwise, they are silently skipped.

Step 4 — User Overrides Win: If you define your own bean of a type that auto-configuration would create, your bean takes precedence. This is the "user-defined beans always win" principle.

3.2 Conditional Annotations

These annotations are the building blocks of auto-configuration. They control when beans are created based on various conditions.

Annotation	Condition	Example
<code>@ConditionalOnClass</code>	A specific class is on the classpath	<code>DataSource.class</code> is present
<code>@ConditionalOnMissingBean</code>	No bean of a type exists in the context	User hasn't defined their own <code>DataSource</code>
<code>@ConditionalOnProperty</code>	A specific property has a certain value	<code>spring.datasource.url</code> is set
<code>@ConditionalOnBean</code>	A specific bean exists in the context	<code>EntityManagerFactory</code> bean is present
<code>@ConditionalOnMissingClass</code>	A specific class is NOT on the classpath	Used for fallback configurations
<code>@ConditionalOnWebApplication</code>	The app is a web application	Servlet-based context detected
<code>@ConditionalOnExpression</code>	A SpEL expression evaluates to true	Complex conditional logic

3.3 Example: DataSource Auto-Configuration

Let's trace how Spring Boot auto-configures a DataSource when you add spring-boot-starter-data-jpa and configure a database URL. This real-world example illustrates the auto-configuration mechanism.

You add a database dependency (e.g., H2 or MySQL driver) and set the URL in application.properties. Spring Boot detects DataSource.class on the classpath. The DataSourceAutoConfiguration class activates because @ConditionalOnClass(DataSource.class) is satisfied. Since you haven't defined your own DataSource bean, @ConditionalOnMissingBean allows Spring Boot to create one using HikariCP (the default connection pool). The DataSource is configured using properties you provided (spring.datasource.url, username, password).

```
# application.properties
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root
spring.datasource.password=secret
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# Spring Boot automatically:
# 1. Creates a HikariCP DataSource bean
# 2. Configures connection pooling defaults
# 3. Sets up EntityManagerFactory if JPA is on classpath
# 4. Creates a PlatformTransactionManager
```

3.4 Debugging Auto-Configuration

When auto-configuration doesn't behave as expected, Spring Boot provides tools to understand what was configured and why.

```
# Enable the auto-configuration report
# Add to application.properties:
debug=true

# This prints a detailed report at startup showing:
# Positive matches - configurations that WERE applied
# Negative matches - configurations that were NOT applied (and why)
# Unconditional classes - configurations applied without conditions
```

Pro Tip for Debugging



Run your application with --debug flag (java -jar myapp.jar --debug) or set debug=true in application.properties. The auto-configuration report in the console will show you exactly which auto-configurations were applied and which were skipped, along with the exact condition that caused the decision.

Interview Q: How does Spring Boot auto-configuration work?

A: Spring Boot auto-configuration works by examining the classpath, existing beans, and property values at startup. It loads auto-configuration classes listed in META-INF/spring/...AutoConfiguration.imports. Each class uses `@Conditional` annotations (e.g., `@ConditionalOnClass`, `@ConditionalOnMissingBean`) to decide if its beans should be created. User-defined beans always take precedence over auto-configured ones. You can debug auto-configuration by setting `debug=true` to see the conditions evaluation report.

4. Starter Dependencies

4.1 What Are Starters?

Starters are a set of convenient dependency descriptors that you can include in your application. They are essentially curated collections of related dependencies that are pre-tested for compatibility. Each starter follows the naming convention `spring-boot-starter-*` where `*` describes the application type or feature.

Real-World Analogy: Consider ordering a meal combo at a restaurant versus ordering individual items. A "burger combo" (starter) includes a burger, fries, and a drink — items that are known to go well together. Similarly, `spring-boot-starter-web` is a "web development combo" that includes Spring MVC, Tomcat, Jackson, and validation libraries — all the ingredients for a web application.

4.2 Common Spring Boot Starters

Starter	What It Includes	Use Case
<code>spring-boot-starter-web</code>	Spring MVC, Tomcat, Jackson, Validation	Building REST APIs and web apps
<code>spring-boot-starter-data-jpa</code>	Hibernate, Spring Data JPA, HikariCP	Database access with JPA/ORM
<code>spring-boot-starter-security</code>	Spring Security, Authentication & Authorization	Securing applications
<code>spring-boot-starter-test</code>	JUnit 5, Mockito, Spring Test, AssertJ	Testing applications
<code>spring-boot-starter-validation</code>	Hibernate Validator, Jakarta Validation API	Input validation
<code>spring-boot-starter-actuator</code>	Metrics, Health checks, Monitoring endpoints	Production monitoring
<code>spring-boot-starter-mail</code>	JavaMail, Spring mail integration	Sending emails
<code>spring-boot-starter-cache</code>	Spring Cache abstraction + providers	Application caching
<code>spring-boot-starter-aop</code>	Spring AOP, AspectJ	Aspect-Oriented Programming
<code>spring-boot-starter-webflux</code>	Reactive web framework (Netty)	Reactive / non-blocking APIs
<code>spring-boot-starter-thymeleaf</code>	Thymeleaf template engine	Server-side HTML rendering
<code>spring-boot-starter-oauth2-client</code>	OAuth 2.0 client support	SSO, social login

4.3 How Starters Work Internally

A starter POM typically contains no code itself. It is a POM file with a set of transitive dependencies. When you add a starter to your project, Maven or Gradle resolves all transitive dependencies and adds them to your classpath.

For example, when you add `spring-boot-starter-web`, it pulls in: `spring-web` and `spring-webmvc` (Spring's web framework), `spring-boot-starter-tomcat` (embedded Tomcat), `spring-boot-starter-json` (Jackson for JSON processing), and `spring-boot-starter` (core starter with Spring Boot auto-configuration, logging, and YAML support).

Interview Q: What are Spring Boot Starters? Can you name some?

A: Starters are curated sets of dependencies bundled together for specific use cases. They follow the naming pattern `spring-boot-starter-*`. They simplify dependency management by providing pre-tested, compatible library sets. Key examples include `spring-boot-starter-web` (REST/web apps), `spring-boot-starter-data-jpa` (database with JPA), `spring-boot-starter-security` (authentication & authorization), `spring-boot-starter-test` (testing), and `spring-boot-starter-actuator` (monitoring). Starters themselves contain no code — just dependency descriptors.

5. Configuration and Properties

5.1 application.properties vs application.yml

Spring Boot allows you to externalize your configuration so that you can work with the same application code in different environments. Configuration can be defined in `application.properties` (flat key-value pairs) or `application.yml` (YAML format with hierarchical structure). Both are placed in the `src/main/resources` directory.

Feature	<code>application.properties</code>	<code>application.yml</code>
Format	key=value (flat)	Hierarchical (indented)
Readability	Good for simple configs	Better for complex/nested configs
Duplicate Keys	Last value wins	Not allowed (YAML spec)
Multi-line Values	Use \ for line continuation	Native support with or >
Profiles	Separate files per profile	Multi-document in single file (---)
List Representation	property[0]=val, property[1]=val	Native YAML list syntax (- val)

Here is a side-by-side comparison of the same configuration in both formats:

application.properties format:

```
server.port=8081
server.servlet.context-path=/api
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root
spring.datasource.password=secret
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
app.name=My Application
app.description=A Spring Boot Training App
```

application.yml format:

```
server:
  port: 8081
  servlet:
    context-path: /api

spring:
  datasource:
    url: jdbc:mysql://localhost:3306/mydb
    username: root
    password: secret
  jpa:
```

```
hibernate:
  ddl-auto: update
  show-sql: true

app:
  name: My Application
  description: A Spring Boot Training App
```

5.2 Injecting Configuration Values

Spring Boot provides multiple ways to inject configuration values into your application code. The most common approaches are `@Value` and `@ConfigurationProperties`.

Using `@Value` (Simple Properties)

```
@RestController
public class AppInfoController {

    @Value("${app.name}")
    private String appName;

    @Value("${app.description:Default Description}")
    private String appDescription; // with default value

    @Value("${server.port}")
    private int serverPort;

    @GetMapping("/info")
    public String getInfo() {
        return appName + " running on port " + serverPort;
    }
}
```

Using `@ConfigurationProperties` (Type-Safe, Recommended)

```
// Step 1: Create a properties class
@Component
@ConfigurationProperties(prefix = "app")
public class AppProperties {

    private String name;
    private String description;
    private Security security = new Security();

    // Getters and Setters required
    public String getName() { return name; }
```

```

    public void setName(String name) { this.name = name; }
    public String getDescription() { return description; }
    public void setDescription(String desc) { this.description = desc; }
    public Security getSecurity() { return security; }

    public static class Security {
        private String apiKey;
        private int tokenExpiry = 3600; // default value
        // Getters and Setters
    }
}

// Step 2: Use it anywhere via injection
@Service
public class MyService {
    private final AppProperties appProperties;

    public MyService(AppProperties appProperties) {
        this.appProperties = appProperties;
    }

    public String getAppInfo() {
        return appProperties.getName();
    }
}

```

Interview Q: What is the difference between @Value and @ConfigurationProperties?

A: @Value injects individual properties one at a time and is suitable for simple cases. @ConfigurationProperties binds an entire group of related properties to a Java object in a type-safe manner. @ConfigurationProperties supports validation (with @Validated), relaxed binding (kebab-case to camelCase), nested objects, and is the recommended approach for complex configuration. @Value does not support meta-data generation for IDE auto-completion, while @ConfigurationProperties does.

5.3 Profiles

Spring Profiles allow you to segregate parts of your application configuration and make them available only in certain environments. This is essential for managing configurations across development, testing, staging, and production environments.

Defining Profile-Specific Properties

```

# application-dev.properties (Development profile)
server.port=8080
spring.datasource.url=jdbc:h2:mem:devdb
spring.jpa.show-sql=true
logging.level.root=DEBUG

```

```
# application-prod.properties (Production profile)
server.port=80
spring.datasource.url=jdbc:mysql://prod-server:3306/proddb
spring.jpa.show-sql=false
logging.level.root=WARN
```

Activating Profiles

```
# Method 1: In application.properties
spring.profiles.active=dev

# Method 2: Command line argument
java -jar myapp.jar --spring.profiles.active=prod

# Method 3: Environment variable
SPRING_PROFILES_ACTIVE=prod java -jar myapp.jar

# Method 4: JVM system property
java -Dspring.profiles.active=prod -jar myapp.jar

# Method 5: Multiple profiles
spring.profiles.active=prod,cloud,metrics
```

Using @Profile on Beans

```
@Configuration
public class DataSourceConfig {

    @Bean
    @Profile("dev")
    public DataSource devDataSource() {
        // H2 in-memory database for development
        return new EmbeddedDatabaseBuilder()
            .setType(EmbeddedDatabaseType.H2)
            .build();
    }

    @Bean
    @Profile("prod")
    public DataSource prodDataSource() {
        // Production MySQL database
        HikariDataSource ds = new HikariDataSource();
        ds.setJdbcUrl("jdbc:mysql://prod-server:3306/proddb");
        return ds;
    }
}
```

Interview Q: What are Spring Profiles and how do you use them?

A: Spring Profiles provide a way to segregate application configuration for different environments (dev, test, staging, prod). You create profile-specific property files (application-{profile}.properties) or annotate beans with `@Profile("profileName")`. Profiles are activated via `spring.profiles.active` property, command-line arguments, environment variables, or JVM properties. Multiple profiles can be active simultaneously. Common uses include different database URLs, logging levels, feature toggles, and service endpoints per environment.

5.4 Property Source Priority (Externalized Configuration)

Spring Boot loads properties from multiple sources in a specific order of precedence. Understanding this order is crucial because a property from a higher-priority source overrides the same property from a lower-priority source.

Priority	Source	Example
1 (Highest)	Command-line arguments	<code>--server.port=9090</code>
2	<code>SPRING_APPLICATION_JSON</code>	Inline JSON in environment variable
3	ServletConfig / ServletContext parameters	Web server init params
4	JNDI attributes	<code>java:comp/env</code> entries
5	Java System properties	<code>-Dserver.port=9090</code>
6	OS Environment variables	<code>SERVER_PORT=9090</code>
7	Profile-specific properties (outside JAR)	<code>config/application-prod.properties</code>
8	Profile-specific properties (inside JAR)	<code>application-prod.properties</code> in classpath
9	Application properties (outside JAR)	<code>config/application.properties</code>
10 (Lowest)	Application properties (inside JAR)	<code>application.properties</code> in classpath

Interview Q: What is the order of property source precedence in Spring Boot?

A: Spring Boot loads properties from multiple sources with a defined precedence. From highest to lowest: command-line arguments, `SPRING_APPLICATION_JSON`, servlet parameters, JNDI, Java System properties (-D), OS environment variables, profile-specific properties (outside JAR first, then inside), and finally `application.properties/yml` inside the JAR. Higher-priority sources override lower ones. This allows runtime overrides without changing packaged configuration.

6. Dependency Injection and IoC Container

6.1 Understanding IoC and DI

Inversion of Control (IoC) is a design principle where the control of object creation and lifecycle management is transferred from the application code to a framework or container. Dependency Injection (DI) is the specific implementation pattern of IoC where dependencies are "injected" into objects rather than being created by the objects themselves.

Real-World Analogy: Consider a restaurant. Without IoC, a chef (your class) goes to the market, buys ingredients, prepares them, and cooks the dish. With IoC, the restaurant manager (Spring Container) procures all ingredients and delivers them to the chef's station. The chef only focuses on cooking (business logic). The chef doesn't know or care where the ingredients came from — they just arrive ready to use.

6.2 Spring IoC Container

The Spring IoC Container is responsible for instantiating, configuring, and assembling objects (called "beans"). It reads configuration metadata (annotations, Java config, or XML) and creates a fully configured application system. Spring provides two types of containers:

Container	Interface	Description
BeanFactory	BeanFactory	Basic container with lazy initialization. Lightweight, minimal features.
ApplicationContext	ApplicationContext	Advanced container that extends BeanFactory. Adds event publishing, internationalization, AOP integration, and eager initialization. This is what Spring Boot uses.

6.3 Types of Dependency Injection

Constructor Injection (Recommended)

```
@Service
public class OrderService {

    private final OrderRepository orderRepository;
    private final PaymentService paymentService;
    private final NotificationService notificationService;
```

```
// Constructor injection - all dependencies are final and immutable
// @Autowired is optional when there is only one constructor
public OrderService(OrderRepository orderRepository,
                    PaymentService paymentService,
                    NotificationService notificationService) {
    this.orderRepository = orderRepository;
    this.paymentService = paymentService;
    this.notificationService = notificationService;
}

public Order placeOrder(Order order) {
    paymentService.processPayment(order);
    Order saved = orderRepository.save(order);
    notificationService.sendConfirmation(saved);
    return saved;
}
}
```

Setter Injection

```
@Service
public class ReportService {

    private EmailService emailService;

    @Autowired
    public void setEmailService(EmailService emailService) {
        this.emailService = emailService;
    }
}
// Use setter injection for optional dependencies
```

Field Injection (Not Recommended)

```
@Service
public class UserService {

    @Autowired // Avoid this pattern
    private UserRepository userRepository;

    // Problems: Cannot make field final, hard to test,
    // hides dependencies, breaks immutability
}
```

Injection Type	Pros	Cons	When to Use
Constructor	Immutable, testable, clear dependencies, fails fast	Verbose with many dependencies	Default choice — use for all mandatory dependencies
Setter	Good for optional dependencies, can change at runtime	Mutable, can be in incomplete state	Optional or reconfigurable dependencies
Field	Concise, less boilerplate	Untestable without reflection, hides dependencies	Avoid in production code; acceptable in test classes

Interview Q: What are the types of Dependency Injection in Spring? Which is preferred and why?

A: Spring supports three types of DI: Constructor Injection (recommended), Setter Injection, and Field Injection. Constructor injection is preferred because it ensures immutability (fields can be final), makes dependencies explicit, guarantees a fully initialized object, and enables easy unit testing without Spring context. As of Spring 4.3, `@Autowired` is optional on single-constructor classes. Setter injection is for optional dependencies. Field injection is discouraged because it hides dependencies and makes unit testing difficult.

6.4 Stereotype Annotations

Spring provides specialized annotations (stereotype annotations) to indicate the role of a class. All are specializations of `@Component` and are detected by component scanning.

Annotation	Layer	Purpose	Special Behavior
<code>@Component</code>	Generic	General-purpose Spring-managed bean	Base annotation; detected by <code>@ComponentScan</code>
<code>@Controller</code>	Presentation	MVC controller for web requests	Returns view names; used with <code>@RequestMapping</code>
<code>@RestController</code>	Presentation	REST API controller	<code>@Controller</code> + <code>@ResponseBody</code> ; returns JSON/XML directly
<code>@Service</code>	Business	Service layer business logic	No additional behavior; semantic marker for clarity
<code>@Repository</code>	Persistence	Data access (DAO) layer	Enables automatic exception translation (SQL to Spring exceptions)

Annotation	Layer	Purpose	Special Behavior
@Configuration	Infrastructure	Java-based configuration class	Processed by CGLIB; @Bean methods define bean definitions

Interview Q: What is the difference between @Component, @Service, @Repository, and @Controller?

A: All are specializations of @Component and are auto-detected by component scanning. @Component is a generic stereotype. @Service marks business logic classes (no special technical behavior, just semantic clarity). @Repository marks data access classes and enables automatic exception translation from database-specific exceptions to Spring's DataAccessException hierarchy. @Controller marks MVC controllers for handling web requests. @RestController combines @Controller and @ResponseBody for REST APIs. Using the correct stereotype improves code readability and enables layer-specific processing.

6.5 Bean Scopes

Bean scope defines the lifecycle and visibility of a bean within the Spring container. Understanding scopes is important for proper application design and resource management.

Scope	Description	Lifecycle	Default?
singleton	One instance per Spring IoC container	Created at startup, destroyed at shutdown	Yes — default scope
prototype	New instance every time the bean is requested	Created on each request; Spring does NOT manage destruction	No
request	One instance per HTTP request (web only)	Created per HTTP request; destroyed when request completes	No
session	One instance per HTTP session (web only)	Created per session; destroyed when session expires	No
application	One instance per ServletContext (web only)	Similar to singleton but scoped to ServletContext	No

```
// Singleton (default)
@Service
public class UserService { } // One instance for entire app

// Prototype
@Component
@Scope("prototype")
public class ShoppingCart { } // New instance each time

// Request scope (web only)
```

```
@Component
@Scope(value = WebApplicationContext.SCOPE_REQUEST,
        proxyMode = ScopedProxyMode.TARGET_CLASS)
public class RequestLogger { } // New instance per HTTP request
```

Interview Q: What are the different bean scopes in Spring Boot?

A: Spring Boot supports five bean scopes. Singleton (default) creates one instance per container, shared across all injection points. Prototype creates a new instance every time the bean is requested. Request, Session, and Application scopes are web-specific: request creates one per HTTP request, session creates one per user session, and application creates one per ServletContext. Singleton is the default and is suitable for stateless services. Prototype is used for stateful beans like shopping carts.

7. Essential Spring Boot Annotations

Spring Boot relies heavily on annotations for configuration, component detection, and feature activation. This chapter provides a comprehensive reference of the most important annotations organized by category.

7.1 Core/Bootstrap Annotations

Annotation	Purpose
@SpringBootApplication	Entry point. Combines @SpringBootConfiguration + @EnableAutoConfiguration + @ComponentScan
@EnableAutoConfiguration	Tells Spring Boot to auto-configure beans based on classpath and properties
@ComponentScan	Configures component scanning directives (packages to scan)
@SpringBootConfiguration	Indicates that a class provides Spring Boot application @Configuration

7.2 Bean Definition & Configuration Annotations

Annotation	Purpose
@Bean	Declares a method that returns an object to be managed as a Spring bean; used inside @Configuration classes
@Configuration	Marks a class as a source of bean definitions; processed using CGLIB proxying
@Autowired	Requests Spring to inject a dependency; optional on single-constructor classes (Spring 4.3+)
@Qualifier	Disambiguates when multiple beans of the same type exist (e.g., @Qualifier("emailNotification"))
@Primary	Marks a bean as the default choice when multiple candidates exist for injection
@Lazy	Bean is created on first use instead of at startup; can be applied to @Bean or @Component
@Scope	Defines the lifecycle scope of a bean: singleton, prototype, request, session, application
@Value	Injects a value from properties, environment variables, or SpEL expressions

Annotation	Purpose
@ConfigurationProperties	Binds a group of properties with a common prefix to a Java object (type-safe)
@PropertySource	Specifies additional property files to load beyond application.properties
@Profile	Restricts a bean or configuration to specific profiles (e.g., dev, prod)
@Conditional	Registers a bean only when a condition is met (base for all @ConditionalOn* annotations)
@Order	Defines the sort order for beans (lower values = higher priority)
@DependsOn	Ensures specific beans are created before the annotated bean

7.3 Web/REST Annotations

Annotation	Purpose
@RestController	@Controller + @ResponseBody; returns data directly (JSON/XML) instead of view names
@Controller	Marks a class as a web MVC controller that returns view names
@RequestMapping	Maps HTTP requests to handler methods; can specify path, method, consumes, produces
@GetMapping	Shortcut for @RequestMapping(method = GET)
@PostMapping	Shortcut for @RequestMapping(method = POST)
@PutMapping	Shortcut for @RequestMapping(method = PUT)
@DeleteMapping	Shortcut for @RequestMapping(method = DELETE)
@PatchMapping	Shortcut for @RequestMapping(method = PATCH)
@PathVariable	Extracts a value from the URI path (e.g., /users/{id})
@RequestParam	Extracts a query parameter (e.g., /users?name=John)
@RequestBody	Binds the HTTP request body to a Java object (deserialization)
@ResponseBody	Indicates the return value should be written to the response body (serialization)
@ResponseStatus	Sets the HTTP status code for the response (e.g., @ResponseStatus(HttpStatus.CREATED))

Annotation	Purpose
@CrossOrigin	Enables Cross-Origin Resource Sharing (CORS) for specific controllers or methods
@ExceptionHandler	Handles exceptions thrown by controller methods
@ControllerAdvice	Global exception handling across all controllers; also for global @ModelAttribute
@RestControllerAdvice	@ControllerAdvice + @ResponseBody for REST APIs

7.4 Validation Annotations

Annotation	Purpose
@Valid / @Validated	Triggers validation on a method parameter or field
@NotNull	Field must not be null
@NotEmpty	String/Collection must not be null or empty
@NotBlank	String must not be null, empty, or only whitespace
@Size(min, max)	String length or Collection size must be within bounds
@Min / @Max	Numeric value must be >= min or <= max
@Email	Must be a well-formed email address
@Pattern(regex)	Must match the regular expression
@Past / @Future	Date must be in the past / future
@Positive / @Negative	Number must be positive / negative

7.5 JPA & Data Annotations (Preview)

While JPA and Spring Data JPA will be covered in detail in a separate document, here is a quick reference of key annotations you will encounter frequently.

Annotation	Purpose
@Entity	Marks a class as a JPA entity (mapped to a database table)
@Table	Specifies the table name if different from the class name
@Id	Marks the primary key field
@GeneratedValue	Defines the primary key generation strategy (AUTO, IDENTITY, SEQUENCE, TABLE)

Annotation	Purpose
@Column	Customizes the column mapping (name, nullable, length, unique)
@Transactional	Marks a method or class for transaction management
@Repository	Marks a DAO class; enables exception translation
@Query	Defines custom JPQL or native SQL queries on repository methods

7.6 Testing Annotations

Annotation	Purpose
@SpringBootTest	Loads full application context for integration testing
@WebMvcTest	Loads only the web layer (controllers) for focused MVC testing
@DataJpaTest	Loads only JPA components for repository testing with in-memory DB
@MockBean	Creates and injects a Mockito mock into the Spring context
@Test	JUnit 5 annotation marking a method as a test case
@BeforeEach / @AfterEach	Setup and teardown methods for each test
@TestPropertySource	Overrides properties for testing

8. Embedded Servers

8.1 Understanding Embedded Servers

One of Spring Boot's most significant features is its embedded server capability. Instead of deploying your application as a WAR file to an external application server (like standalone Tomcat, WildFly, or WebSphere), Spring Boot packages an entire web server inside your application JAR. When you run the JAR, the embedded server starts automatically.

Real-World Analogy: Traditional deployment is like shipping furniture that the customer must assemble (IKEA style). Embedded servers are like shipping fully assembled furniture — it arrives ready to use. This self-contained approach is particularly powerful for microservices and cloud-native applications where each service needs to be independently deployable.

8.2 Supported Embedded Servers

Server	Type	Starter	When to Choose
Tomcat	Servlet-based	spring-boot-starter-web (default)	General purpose; most widely used; large community
Jetty	Servlet-based	Replace Tomcat with spring-boot-starter-jetty	Long-lived connections; WebSocket-heavy apps; lighter footprint
Undertow	Servlet-based	Replace Tomcat with spring-boot-starter-undertow	High performance; non-blocking I/O; JBoss ecosystem
Netty	Reactive (non-servlet)	spring-boot-starter-webflux (default)	Reactive/non-blocking applications; high concurrency

Switching from Tomcat to Jetty

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
  <exclusions>
    <exclusion>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-tomcat</artifactId>
    </exclusion>
  </exclusions>
</dependency>
```

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-jetty</artifactId>
</dependency>
```

8.3 Server Configuration Properties

```
# Common server configurations
server.port=8081                # Change default port (8080)
server.port=0                   # Random available port
server.address=0.0.0.0          # Bind to all interfaces
server.servlet.context-path=/api/v1 # Context path prefix

# SSL / HTTPS
server.ssl.enabled=true
server.ssl.key-store=classpath:keystore.p12
server.ssl.key-store-password=changeit
server.ssl.key-store-type=PKCS12

# Connection tuning
server.tomcat.max-threads=200   # Max worker threads
server.tomcat.accept-count=100  # Queue size when threads are busy
server.tomcat.max-connections=8192 # Max concurrent connections
server.tomcat.connection-timeout=20000 # Connection timeout (ms)

# Compression
server.compression.enabled=true
server.compression.min-response-size=1024 # Compress responses > 1KB
```

Interview Q: What is an embedded server in Spring Boot? How do you change the default server?

A: Spring Boot embeds a web server (Tomcat by default) directly inside the application JAR, eliminating the need for an external application server. You run the app with `java -jar` and the server starts automatically. To switch servers, exclude the default Tomcat starter and add an alternative (`spring-boot-starter-jetty` or `spring-boot-starter-undertow`). Common configurations include `server.port` to change the port, `server.servlet.context-path` for URL prefix, and `server.ssl.*` for HTTPS.

9. Spring Boot Actuator

9.1 What is Actuator?

Spring Boot Actuator provides production-ready features to help you monitor and manage your application. It exposes operational information about the running application through HTTP endpoints or JMX beans. This includes health status, metrics, environment details, bean information, and much more.

Adding Actuator to Your Project

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

9.2 Key Actuator Endpoints

Endpoint	URL	Purpose
health	/actuator/health	Application health status (UP/DOWN); integrates with DB, disk, custom checks
info	/actuator/info	Custom application information (from info.* properties)
metrics	/actuator/metrics	Application metrics (JVM memory, CPU, HTTP requests, custom counters)
env	/actuator/env	All environment properties and their sources
beans	/actuator/beans	Complete list of all Spring beans in the application context
mappings	/actuator/mappings	All @RequestMapping paths and their handler methods
configprops	/actuator/configprops	All @ConfigurationProperties beans and their values
loggers	/actuator/loggers	View and dynamically change logging levels at runtime
threaddump	/actuator/threaddump	Current thread dump for debugging
heapdump	/actuator/heapdump	Heap dump file download for memory analysis

Endpoint	URL	Purpose
httptrace	/actuator/httptrace	Recent HTTP request-response exchanges
shutdown	/actuator/shutdown	Graceful shutdown (disabled by default; POST only)

9.3 Configuring Actuator

```
# Expose specific endpoints over HTTP
management.endpoints.web.exposure.include=health,info,metrics,env

# Expose ALL endpoints (use with caution in production)
management.endpoints.web.exposure.include=*

# Hide sensitive endpoints
management.endpoints.web.exposure.exclude=heapdump,threaddump

# Show full health details
management.endpoint.health.show-details=always

# Custom management port (separate from app port)
management.server.port=9090

# Custom base path
management.endpoints.web.base-path=/manage

# Enable shutdown endpoint
management.endpoint.shutdown.enabled=true

# Add application info
info.app.name=My Application
info.app.version=1.0.0
info.app.description=Spring Boot Training Application
```

9.4 Custom Health Indicators

You can create custom health indicators to check the status of external dependencies like third-party APIs, message brokers, or custom infrastructure.

```
@Component
public class PaymentGatewayHealthIndicator implements HealthIndicator {

    private final PaymentGatewayClient paymentClient;

    public PaymentGatewayHealthIndicator(PaymentGatewayClient client) {
```

```
        this.paymentClient = client;
    }

    @Override
    public Health health() {
        try {
            boolean isReachable = paymentClient.ping();
            if (isReachable) {
                return Health.up()
                    .withDetail("provider", "Stripe")
                    .withDetail("status", "Reachable")
                    .build();
            }
            return Health.down()
                .withDetail("error", "Payment gateway not responding")
                .build();
        } catch (Exception e) {
            return Health.down(e).build();
        }
    }
}
```

Interview Q: What is Spring Boot Actuator and what are its key endpoints?

A: Spring Boot Actuator provides production-ready monitoring and management features via HTTP endpoints or JMX. Key endpoints include `/health` (application health status), `/metrics` (JVM and app metrics), `/env` (environment properties), `/beans` (all registered beans), `/mappings` (URL mappings), `/loggers` (view/change log levels at runtime), and `/info` (custom app information). Endpoints are configurable via `management.endpoints.web.exposure.include`. You can create custom health indicators by implementing the `HealthIndicator` interface.

10. Logging in Spring Boot

10.1 Default Logging Framework

Spring Boot uses SLF4J (Simple Logging Facade for Java) as the logging abstraction and Logback as the default implementation. This combination provides a powerful, flexible logging system out of the box. SLF4J acts as a facade, meaning you program against the SLF4J API and the actual logging implementation (Logback, Log4j2, etc.) is pluggable.

Using Logging in Your Code

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@Service
public class OrderService {

    private static final Logger log = LoggerFactory.getLogger(OrderService.class);

    public Order placeOrder(Order order) {
        log.info("Placing order for customer: {}", order.getCustomerId());
        log.debug("Order details: {}", order);

        try {
            // process order
            log.info("Order {} placed successfully", order.getId());
        } catch (Exception e) {
            log.error("Failed to place order for customer {}",
                    order.getCustomerId(), e);
        }
        return order;
    }
}
```

10.2 Log Levels

Level	Priority	When to Use
TRACE	Lowest	Extremely detailed diagnostic info; method entry/exit points
DEBUG	Low	Diagnostic information useful during development; variable values
INFO	Medium	General operational messages; application milestones; startup info

Level	Priority	When to Use
WARN	High	Potentially harmful situations; recoverable issues; deprecation notices
ERROR	Highest	Error events; exceptions; failures that need attention

Configuring Log Levels

```
# Set root log level
logging.level.root=WARN

# Set package-specific levels
logging.level.com.example.myapp=DEBUG
logging.level.com.example.myapp.repository=TRACE
logging.level.org.springframework.web=INFO
logging.level.org.hibernate.SQL=DEBUG

# Log to file
logging.file.name=logs/application.log

# OR log to directory (auto-named spring.log)
logging.file.path=/var/logs/myapp

# Custom log pattern
logging.pattern.console=%d{yyyy-MM-dd HH:mm:ss} [%thread] %-5level %logger{36} - %msg%n
logging.pattern.file=%d{yyyy-MM-dd HH:mm:ss} [%thread] %-5level %logger{36} - %msg%n
```

Best Practice for Logging



Use parameterized messages (`log.info("User {} logged in", username)`) instead of string concatenation (`log.info("User " + username + " logged in")`). Parameterized logging avoids the cost of string concatenation when the log level is disabled. Also, never log sensitive data like passwords, credit card numbers, or personal identification information.

Interview Q: What logging framework does Spring Boot use by default?

A: Spring Boot uses SLF4J as the logging facade and Logback as the default implementation. SLF4J provides a vendor-neutral API, allowing you to switch implementations (e.g., to Log4j2) without changing application code. Log levels from lowest to highest are TRACE, DEBUG, INFO, WARN, ERROR. Levels are configured via `logging.level.*` properties. Spring Boot also supports file logging, custom patterns, and integration with Actuator for runtime log level changes via `/actuator/loggers`.

11. Exception Handling

11.1 Global Exception Handling with @ControllerAdvice

Spring Boot provides a powerful mechanism for centralized exception handling using `@ControllerAdvice` (or `@RestControllerAdvice` for REST APIs). This eliminates the need to handle exceptions in every controller and provides consistent error responses across the application.

Step 1: Define Custom Exception Classes

```
// Custom business exception
public class ResourceNotFoundException extends RuntimeException {
    private final String resourceName;
    private final String fieldName;
    private final Object fieldValue;

    public ResourceNotFoundException(String resourceName,
                                    String fieldName, Object fieldValue) {
        super(String.format("%s not found with %s: '%s'",
                            resourceName, fieldName, fieldValue));
        this.resourceName = resourceName;
        this.fieldName = fieldName;
        this.fieldValue = fieldValue;
    }

    // Getters
}
```

Step 2: Create a Standard Error Response DTO

```
public class ApiErrorResponse {
    private LocalDateTime timestamp;
    private int status;
    private String error;
    private String message;
    private String path;
    private Map<String, String> validationErrors;

    // Constructors, Getters, Setters
}
```

Step 3: Create the Global Exception Handler

```
@RestControllerAdvice
public class GlobalExceptionHandler {
```

```

    @ExceptionHandler(ResourceNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ApiErrorResponse handleResourceNotFound(
        ResourceNotFoundException ex, WebRequest request) {
        return new ApiErrorResponse(
            LocalDateTime.now(),
            HttpStatus.NOT_FOUND.value(),
            "Not Found",
            ex.getMessage(),
            request.getDescription(false)
        );
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ApiErrorResponse handleValidationErrors(
        MethodArgumentNotValidException ex) {
        Map<String, String> errors = new HashMap<>();
        ex.getBindingResult().getFieldErrors()
            .forEach(err -> errors.put(
                err.getField(), err.getDefaultMessage()));

        ApiErrorResponse response = new ApiErrorResponse(
            LocalDateTime.now(),
            400, "Validation Failed",
            "Input validation failed", null);
        response.setValidationErrors(errors);
        return response;
    }

    @ExceptionHandler(Exception.class)
    @ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
    public ApiErrorResponse handleAllOther(Exception ex) {
        return new ApiErrorResponse(
            LocalDateTime.now(),
            500, "Internal Server Error",
            "An unexpected error occurred", null);
    }
}

```

Interview Q: How do you handle exceptions globally in Spring Boot?

A: You use `@RestControllerAdvice` (or `@ControllerAdvice` for MVC) to create a centralized exception handler class. Inside, you define methods annotated with `@ExceptionHandler` specifying the exception types to handle. Each handler method returns a standardized error response DTO with details like timestamp, status code, error message, and path. This pattern ensures consistent error responses across all controllers. For validation errors, you handle `MethodArgumentNotValidException` to return field-level error details.

12. Spring Boot DevTools

12.1 What is DevTools?

Spring Boot DevTools is a module that enhances the development experience by providing features like automatic restart, live reload, relaxed property defaults, and remote debugging. It is designed to be used only during development and is automatically disabled in production.

Adding DevTools

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
  <scope>runtime</scope>
  <optional>true</optional>
</dependency>
```

12.2 Key Features

Feature	Description	Benefit
Automatic Restart	Restarts the application when classpath files change	No manual restart; changes visible in seconds
Live Reload	Triggers browser refresh when resources change	Instant feedback for static resource changes
Property Defaults	Sets development-friendly defaults (e.g., disables template caching)	No need to manually configure dev settings
Remote Debugging	Supports remote application restarts and debugging	Debug applications running in remote environments
H2 Console	Auto-enables H2 database web console	Easy database inspection during development

How DevTools Knows It is Development



DevTools is automatically disabled when the application is launched from a fully packaged JAR (java -jar). It uses two classloaders: a "base" classloader for third-party JARs (unchanged) and a "restart" classloader for your project classes. Only the restart classloader is discarded and recreated on changes, making restarts much faster than a full cold start.

13. Building and Deploying Spring Boot Applications

13.1 Executable JAR (Fat JAR / Uber JAR)

Spring Boot's default packaging creates an executable JAR (also called a "fat JAR" or "uber JAR") that contains your compiled code, all dependencies, and an embedded web server. This single JAR file is everything you need to run your application.

```
# Build the JAR
mvn clean package

# The JAR is created in the target/ directory
# target/my-application-1.0.0.jar

# Run the application
java -jar target/my-application-1.0.0.jar

# Run with custom properties
java -jar myapp.jar --server.port=9090 --spring.profiles.active=prod

# Run with JVM options
java -Xmx512m -Xms256m -jar myapp.jar
```

13.2 JAR vs WAR Deployment

Aspect	JAR (Recommended)	WAR
Server	Embedded (Tomcat/Jetty/Undertow)	External application server
Execution	java -jar myapp.jar	Deploy to server's webapps directory
Self-Contained	Yes — everything in one file	No — depends on external server
Cloud-Ready	Yes — ideal for containers/Docker	Requires server in container
Startup	Faster — single process	Slower — server must initialize first
Spring Boot Default	Yes	Requires changes to main class
Best For	Microservices, cloud, Docker, standalone apps	Legacy infrastructure, shared servers

13.3 Docker Deployment

```
# Dockerfile for Spring Boot application
FROM eclipse-temurin:17-jre-alpine
```

```
WORKDIR /app
```

```
COPY target/my-application-1.0.0.jar app.jar
```

```
EXPOSE 8080
```

```
# Use exec form for proper signal handling
```

```
ENTRYPOINT ["java", "-jar", "app.jar"]
```

```
# Build and run
```

```
docker build -t my-spring-app .
```

```
docker run -p 8080:8080 -e SPRING_PROFILES_ACTIVE=prod my-spring-app
```

Interview Q: What is a fat JAR in Spring Boot? How is it different from a WAR?

A: A fat JAR (or uber JAR) is an executable JAR that packages the application code, all dependencies, and an embedded web server into a single file. You run it with `java -jar`. A WAR file requires deployment to an external application server (like standalone Tomcat or WildFly). Spring Boot defaults to JAR packaging because it is self-contained, simpler to deploy, cloud-friendly, and ideal for microservices and containerized environments.

14. Testing Spring Boot Applications

14.1 Testing Strategy Overview

Spring Boot provides excellent testing support through the `spring-boot-starter-test` dependency, which includes JUnit 5, Mockito, AssertJ, Hamcrest, JSONPath, and Spring Test utilities. A well-tested application typically follows the testing pyramid: many unit tests, fewer integration tests, and a small number of end-to-end tests.

14.2 Unit Testing (Service Layer)

```
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {

    @Mock
    private OrderRepository orderRepository;

    @Mock
    private PaymentService paymentService;

    @InjectMocks
    private OrderService orderService;

    @Test
    void placeOrder_ValidOrder_ReturnsCreatedOrder() {
        // Arrange
        Order order = new Order("CUST-001", BigDecimal.valueOf(99.99));
        when(orderRepository.save(any(Order.class)))
            .thenReturn(order);
        when(paymentService.processPayment(any()))
            .thenReturn(true);

        // Act
        Order result = orderService.placeOrder(order);

        // Assert
        assertNotNull(result);
        assertEquals("CUST-001", result.getCustomerId());
        verify(orderRepository).save(any(Order.class));
        verify(paymentService).processPayment(any());
    }
}
```

14.3 Integration Testing with @SpringBootTest

```

@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
class OrderControllerIntegrationTest {

    @Autowired
    private TestRestTemplate restTemplate;

    @Test
    void createOrder_ValidInput_Returns201() {
        OrderRequest request = new OrderRequest("CUST-001", 99.99);

        ResponseEntity<Order> response = restTemplate.postForEntity(
            "/api/orders", request, Order.class);

        assertEquals(HttpStatus.CREATED, response.getStatusCode());
        assertNotNull(response.getBody().getId());
    }
}

```

14.4 Slice Testing

Slice tests load only a portion of the application context, making them faster than full integration tests while still testing Spring configuration.

Annotation	What It Tests	What It Loads
@WebMvcTest	Controllers, filters, advice	Web layer only; mocks service layer
@DataJpaTest	Repositories, entities	JPA layer with in-memory DB; auto-rollback
@WebFluxTest	Reactive controllers	WebFlux layer only
@JsonTest	JSON serialization/deserialization	Jackson ObjectMapper configuration
@RestClientTest	REST client (RestTemplate)	REST client infrastructure

Example: @WebMvcTest

```

@WebMvcTest(OrderController.class)
class OrderControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private OrderService orderService;

    @Test

```

```
void getOrder_ExistingId_ReturnsOrder() throws Exception {  
    Order order = new Order(1L, "CUST-001");  
    when(orderService.findById(1L)).thenReturn(order);  
  
    mockMvc.perform(get("/api/orders/1"))  
        .andExpect(status().isOk())  
        .andExpect(jsonPath("$.customerId").value("CUST-001"));  
}  
}
```

Interview Q: What is the difference between @SpringBootTest, @WebMvcTest, and @DataJpaTest?

A: @SpringBootTest loads the complete application context and is used for full integration testing. @WebMvcTest loads only the web layer (controllers, filters, exception handlers) and mocks the service layer — ideal for testing controller logic. @DataJpaTest loads only the JPA layer (repositories, entities, DataSource) with an embedded in-memory database and auto-rollback — ideal for testing queries. Slice tests (@WebMvcTest, @DataJpaTest) are faster than @SpringBootTest because they load fewer beans.

15. Additional Spring Boot Features

15.1 CommandLineRunner and ApplicationRunner

These interfaces allow you to execute code immediately after the Spring Boot application has started. They are commonly used for data initialization, cache warming, sanity checks, or one-time setup tasks.

```
@Component
public class DataInitializer implements CommandLineRunner {

    private final UserRepository userRepository;

    public DataInitializer(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    @Override
    public void run(String... args) throws Exception {
        if (userRepository.count() == 0) {
            userRepository.save(new User("admin", "admin@example.com"));
            userRepository.save(new User("user", "user@example.com"));
            System.out.println("Sample data initialized.");
        }
    }
}
```

15.2 Event Listeners and Application Events

Spring Boot publishes several events during the application lifecycle. You can listen to these events to perform actions at specific stages.

```
@Component
public class AppEventListener {

    @EventListener(ApplicationReadyEvent.class)
    public void onApplicationReady() {
        System.out.println("Application is ready to serve requests!");
    }

    @EventListener(ContextClosedEvent.class)
    public void onShutdown() {
        System.out.println("Application is shutting down...");
        // Cleanup resources, close connections
    }
}
```

15.3 Scheduling Tasks

```
@SpringBootApplication
@EnableScheduling // Enable scheduling support
public class MyApplication { }

@Component
public class ScheduledTasks {

    @Scheduled(fixedRate = 60000) // Every 60 seconds
    public void reportCurrentTime() {
        log.info("Current time: {}", LocalDateTime.now());
    }

    @Scheduled(cron = "0 0 9 * * MON-FRI") // 9 AM weekdays
    public void sendDailyReport() {
        // Generate and send daily report
    }

    @Scheduled(fixedDelay = 30000, initialDelay = 10000)
    public void cleanupExpiredTokens() {
        // Runs 30s after previous execution completes
        // First run delayed by 10s after startup
    }
}
```

15.4 Caching

```
@SpringBootApplication
@EnableCaching // Enable caching support
public class MyApplication { }

@Service
public class ProductService {

    @Cacheable(value = "products", key = "#id")
    public Product getProductById(Long id) {
        // This method result is cached; subsequent calls
        // with same id return cached result without DB hit
        return productRepository.findById(id).orElseThrow();
    }

    @CacheEvict(value = "products", key = "#product.id")
    public Product updateProduct(Product product) {
        // Evicts the cached entry when product is updated
        return productRepository.save(product);
    }
}
```

```
@CacheEvict(value = "products", allEntries = true)
public void clearProductCache() {
    // Clears all entries in the 'products' cache
}
}
```

15.5 Async Execution

```
@SpringBootApplication
@EnableAsync // Enable async support
public class MyApplication { }

@Service
public class NotificationService {

    @Async
    public CompletableFuture<String> sendEmailAsync(String to, String body) {
        // Executes in a separate thread
        emailClient.send(to, body);
        return CompletableFuture.completedFuture("Email sent to " + to);
    }
}
```

16. Frequently Asked Interview Questions

This section compiles the most commonly asked Spring Boot interview questions with concise, interview-ready answers. These cover conceptual understanding, practical knowledge, and common scenarios.

Interview Q: Can you change the default port of a Spring Boot application?

A: Yes. Set `server.port=8081` in `application.properties` or `application.yml`. You can also use `server.port=0` for a random available port. Command-line override: `java -jar app.jar --server.port=9090`. Environment variable: `SERVER_PORT=9090`.

Interview Q: How does Spring Boot achieve auto-configuration?

A: Spring Boot reads auto-configuration classes from `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`. Each class uses `@Conditional` annotations to check classpath, existing beans, and property values. If conditions are met, beans are auto-created. User-defined beans always take precedence.

Interview Q: What is the difference between `@SpringBootApplication` and `@EnableAutoConfiguration`?

A: `@SpringBootApplication` is a convenience annotation that combines `@EnableAutoConfiguration`, `@ComponentScan`, and `@SpringBootConfiguration`. `@EnableAutoConfiguration` alone only enables auto-configuration without component scanning. In practice, always use `@SpringBootApplication` as it covers all three.

Interview Q: What is Spring Boot Actuator and why is it important?

A: Actuator provides production-ready features: `/health` for application health, `/metrics` for JVM and app metrics, `/env` for configuration, `/loggers` for runtime log level changes, and more. It is critical for monitoring, debugging, and operating applications in production, integrating with tools like Prometheus and Grafana.

Interview Q: How do you handle different environments in Spring Boot?

A: Use Spring Profiles. Create profile-specific property files (`application-dev.properties`, `application-prod.properties`). Activate with `spring.profiles.active=prod` via properties, command-line, environment variables, or JVM args. You can also annotate beans with `@Profile` to create environment-specific beans.

Interview Q: What is the purpose of `spring-boot-starter-parent`?

A: It is a special parent POM that provides dependency management (curated, compatible versions for hundreds of libraries), sensible plugin defaults (compiler settings, resource filtering), and Spring Boot's BOM (Bill of Materials). It eliminates version conflicts and ensures all Spring ecosystem libraries work together.

Interview Q: How does Spring Boot know which beans to create?

A: Through `@ComponentScan` (discovers `@Component`, `@Service`, `@Repository`, `@Controller`), `@Bean` methods in `@Configuration` classes (explicit bean definitions), and `@EnableAutoConfiguration` (auto-creates beans based on classpath analysis and conditional evaluation). All three mechanisms work together at startup.

Interview Q: What is the difference between `@Controller` and `@RestController`?

A: `@Controller` returns view names (for MVC with templates like Thymeleaf). `@RestController` combines `@Controller` and `@ResponseBody`, so every method return value is automatically serialized to JSON/XML and written to the HTTP response body. Use `@RestController` for REST APIs and `@Controller` for server-rendered views.

Interview Q: How do you deploy a Spring Boot application?

A: The standard approach is to package as an executable JAR (`mvn clean package`) and run with `java -jar app.jar`. For containerization, create a Dockerfile with a JRE base image, copy the JAR, and expose the port. For cloud platforms (AWS, Azure, GCP), use their respective deployment mechanisms. WAR deployment to external servers is also supported but less common.

Interview Q: What is the difference between `@Bean` and `@Component`?

A: `@Component` is a class-level annotation that auto-registers the class as a bean via component scanning. `@Bean` is a method-level annotation used inside `@Configuration` classes to manually define a bean. Use `@Bean` when you need to configure third-party classes (that you cannot annotate with `@Component`) or when you need custom initialization logic.

17. Quick Reference Summary

Below is a rapid-reference table summarizing the key Spring Boot concepts covered in this document.

Concept	Key Points
Auto-Configuration	Classpath-driven, @Conditional annotations, user beans win, debug=true for report
Starters	Curated dependency sets, spring-boot-starter-* naming, no version needed with parent POM
Embedded Server	Tomcat (default), Jetty, Undertow, Netty (reactive); configured via server.* properties
Profiles	application-{profile}.properties, spring.profiles.active, @Profile annotation
Properties	@Value for simple, @ConfigurationProperties for type-safe, YAML or properties format
Dependency Injection	Constructor (preferred), Setter, Field; IoC Container manages lifecycle
Stereotype Annotations	@Component, @Service, @Repository, @Controller, @RestController, @Configuration
Bean Scopes	singleton (default), prototype, request, session, application
Actuator	Health, metrics, env, loggers, beans, mappings; configurable exposure
Logging	SLF4J + Logback default; TRACE < DEBUG < INFO < WARN < ERROR
Exception Handling	@RestControllerAdvice + @ExceptionHandler for global, consistent error responses
DevTools	Auto-restart, live reload, dev-friendly defaults; disabled in production
Testing	@SpringBootTest (full), @WebMvcTest (web), @DataJpaTest (JPA), MockMvc, TestRestTemplate
Deployment	Executable JAR (default), WAR (legacy), Docker, Cloud platforms

End of Spring Boot Training Notes
Next Module: JPA, Spring Data JPA, and REST API Development